

第 4 次作业

算法设计与分析

姓名	倪伟丰	学号	2024110089
班级	24 大数据 2 班	教师	韩凯
提交日期	2026 年 3 月 27 日		

1 第 1 题

Quantum antimatter fuel comes in small pellets, which is convenient since the many moving parts of the LAMBCHOP each need to be fed fuel one pellet at a time. However, minions dump pellets in bulk into the fuel intake. You need to figure out the most efficient way to sort and shift the pellets down to a single pellet at a time. The fuel control mechanisms have three operations:

- Add 1 fuel pellet
- Remove 1 fuel pellet
- Divide the entire group of fuel pellets by 2 (due to the destructive energy released when a quantum antimatter pellet is cut in half, the safety controls will only allow this to happen if there is an even number of pellets)

Write a function called `answer(n)` which takes a positive integer n as a string and returns the minimum number of operations needed to transform the number of pellets to 1. Prove the correctness of your algorithm. (E.g. `29to28to14to7to8to4to2to1`)

解:

我们使用贪心算法来求解这个问题。对于当前的燃料数量 $n, n \neq 1$, 我们考虑以下情况:

1. 如果 n 是偶数, 我们可以直接执行除以 2 的操作, 这样可以快速减少燃料数量。
2. 如果 n 是奇数:
 - 如果 $n = 4k + 3, \forall k = 1, 2, 3, \dots$, 我们选择加 1, 这样可以得到一个 4 的倍数, 然后执行 2 次除以 2 的操作。
 - 如果 $n = 4k + 1, \forall k = 1, 2, 3, \dots$, 我们选择减 1, 这样可以得到一个 4 的倍数, 然后执行 2 次除以 2 的操作。
 - 如果 $n = 3$, 我们选择减 1, 这样可以得到 2, 然后执行除以 2 的操作, 两部内达到 1, 如不是先 +1 再执行两次除以 2 的操作 (这需要 3 步)。

解:

由此我们得到以下 function (使用 C++):

```
int answer(string n) {
    int steps = 0;
    long long num = stoll(n);

    while (num != 1) {
        if (num % 2 == 0) {
            num /= 2;
        } else if (num == 3 || num % 4 == 1) {
            num -= 1;
        } else {
            num += 1;
        }
        steps++;
    }

    return steps;
}
```

接下来我们需要证明:

- 引理 1: 对于任何偶数 n , 执行除以 2 的操作是最优的。
- 引理 2: 对于任何奇数 n , 如果 $n = 4k + 3, \forall k = 1, 2, 3, \dots$, 执行加 1 的操作是最优的; 如果 $n = 4k + 1, \forall k = 1, 2, 3, \dots$, 执行减 1 的操作是最优的; 如果 $n = 3$, 执行减 1 的操作是最优的。
- 引理 3: 当 $n = 3$ 时, 执行减 1 的操作是最优的。

将以上引理数学化, 即: 假设 $T(n)$ 表示将 n 转换为 1 所需的最小操作数。于是:

- $T(1) = 0$
- $T(3) = 2$
- $\forall k = 1, 2, 3, \dots$
 - $T(2k) = 1 + T(k)$
 - $T(4k + 1) = 1 + T(4k)$
 - $T(4k + 3) = 1 + T(4k + 4)$

首先证明引理 3, 即: $T(3) = 2$ 显然, $T(3) = \min\{T(2) + 1, T(4) + 1\} = \min\{1 + T(2), 1 + T(4)\} = \min\{1 + 1, 1 + 2\} = 2$

接下来证明引理 1, 即: 对于任何偶数 n , 执行除以 2 的操作是最优的。

$\forall n = 2k, k = 1, 2, 3, \dots, T(n) = 1 + \min\{T(\frac{n}{2}), T(n + 1), T(n - 1)\}$, 要证明引理 1 成立, 即证明:

$$T\left(\frac{n}{2}\right) \leq T(n + 1) \text{ 且}$$

2 第 2 题

Give a topological ordering of the graph in Figure 2.

解:

一个合法的拓扑排序是: $v_6, v_5, v_3, v_1, v_9, v_{10}, v_8, v_7, v_2, v_4$ 。

3 第 3 题

(Exercise 3.10 from the textbook) Suppose we are given an undirected graph $G = (V, E)$, and we identify two nodes v and w in G . Give an algorithm that computes the number of shortest v - w paths in G . (The algorithm should not list all the paths; just the number suffices.) The running time of your algorithm should be $O(m + n)$ for a graph with n nodes and m edges.

解:

我们使用 BFS 统计从 v 到 w 的最短路径条数。

Algorithm 1

```
1: function COUNTSHORTESTPATHS( $G, v, w$ )
2:   for each vertex  $x \in V$  do
3:     distance[ $x$ ]  $\leftarrow \infty$ 
4:     count[ $x$ ]  $\leftarrow 0$ 
5:   end
6:   distance[ $v$ ]  $\leftarrow 0$ 
7:   count[ $v$ ]  $\leftarrow 1$ 
8:   Queue  $Q$ 
9:    $Q.enqueue(v)$ 
10:  while  $Q$  is not empty do
11:     $u \leftarrow Q.dequeue()$ 
12:    for each vertex  $x$  in  $u.adj$  do
13:      if distance[ $x$ ] = infinity then
14:        distance[ $x$ ]  $\leftarrow$  distance[ $u$ ] + 1
15:        count[ $x$ ]  $\leftarrow$  count[ $u$ ]
16:         $Q.enqueue(x)$ 
17:      end
18:      else if distance[ $x$ ] = distance[ $u$ ] + 1 then
19:        count[ $x$ ]  $\leftarrow$  count[ $x$ ] + count[ $u$ ]
20:      end
21:    end
22:  end
23:  return count[ $w$ ]
24: end
```

解:

上述算法的正确性可由 BFS 的分层性质证明。设 $d(x)$ 表示图中从 v 到顶点 x 的最短距离；若 x 不可达，则记为 ∞ 。

首先证明数组 `distance` 是正确的。BFS 会按照与源点距离从小到大的顺序访问各层顶点。当顶点 x 第一次被某个顶点 u 访问到时，算法令 $\text{distance}[x] = \text{distance}[u] + 1$ 。由于此时已经存在一条从 v 到 u 的长度为 $\text{distance}[u]$ 的路径，再接上边 (u, x) 就得到一条从 v 到 x 的长度为 $\text{distance}[u] + 1$ 的路径，因此可得 $d(x) \leq \text{distance}[x]$ 。另一方面，如果存在一条更短的从 v 到 x 的路径，那么这条路径上紧邻 x 的前驱顶点必然会在更早的一层被访问，从而 x 应该更早就被发现，这与“ x 此时第一次被访问到”矛盾。因此对每个可达顶点都有 $\text{distance}[x] = d(x)$ ；对不可达顶点， $\text{distance}[x]$ 始终保持为 ∞ ，也同样正确。

接着证明数组 `count` 是正确的。源点 v 到自身的最短路径只有一条，因此初始化 $\text{count}[v] = 1$ 是正确的。下面按 BFS 层数归纳：假设对所有满足 $\text{distance}[u] \leq k$ 的顶点 u ， $\text{count}[u]$ 都已经等于从 v 到 u 的最短路径条数。考虑任意满足 $\text{distance}[x] = k + 1$ 的顶点 x 。任意一条从 v 到 x 的最短路径，其倒数第二个顶点必为某个邻接点 u ，并且满足 $\text{distance}[u] = k$ ；反过来，任意满足 $\text{distance}[u] = k$ 且与 x 相邻的顶点 u ，都可以把每一条从 v 到 u 的最短路径后接边 (u, x) ，得到一条从 v 到 x 的最短路径。算法第一次发现 x 时，将 $\text{count}[x]$ 设为该前驱的 `count`；之后每遇到一个新的前驱 u 满足 $\text{distance}[x] = \text{distance}[u] + 1$ ，就执行 $\text{count}[x] += \text{count}[u]$ 。因此，所有能够作为最短路径倒数第二个顶点的前驱都会被恰好计入一次，而不能满足该条件的邻点不会被计入。由归纳假设可知，这些加总起来恰好就是从 v 到 x 的全部最短路径条数。

因此，当 BFS 结束时， $\text{count}[w]$ 恰好等于从 v 到 w 的最短路径数；如果 w 从 v 不可达，则 $\text{count}[w]$ 保持为 0，这也与题意一致。

上述算法的时间复杂度为 $O(m + n)$ 。初始化 `distance` 和 `count` 两个数组时，需要对每个顶点访问一次，因此这一部分耗时为 $O(n)$ 。在 BFS 过程中，每个顶点至多入队一次、出队一次，所以所有队列操作的总时间为 $O(n)$ 。同时，我们对每个顶点只遍历一次它的邻接表，因此所有邻接表的遍历总时间与边数成正比；由于图是无向图，每条边至多被它的两个端点各检查一次，因此这一部分总耗时为 $O(m)$ 。综上，算法的总运行时间为 $O(m + n)$ 。

4 第 4 题

(Exercise 3.12 from the textbook) You're helping a group of ethnographers analyze some oral history data they've collected by interviewing members of a village to learn about the lives of people who've lived there over the past two hundred years.

From these interviews, they've learned about a set of n people (all of them now deceased), whom we'll denote $P_1, P_2, \dots, P_n$. They've also collected facts about when these people lived relative to one another. Each fact has one of the following two forms:

- For some i and j , person P_i died before person P_j was born; or
- for some i and j , the life spans of P_i and P_j overlapped at least partially.

Naturally, they're not sure that all these facts are correct; memories are not so good, and a lot of this was passed down by word of mouth. So what they'd like you to determine is whether the data they've collected is at least internally consistent, in the sense that there could have existed a set of people for which all the facts they've learned simultaneously hold. Give an efficient algorithm to do this: either it should produce proposed dates of birth and death for each of the n people so that

all the facts hold true, or it should report (correctly) that no such dates can exist—that is, the facts collected by the ethnographers are not internally consistent.

解:

设 $H = (U, F)$ 为表示全部时间约束的有向图。对每个人 P_i 建立两个节点 B_i 与 D_i ，分别表示其出生事件和死亡事件。若在图中加入一条边 $x \rightarrow y$ ，则表示约束“事件 x 必须严格早于事件 y ”。

建图规则如下：

1. 对每个 i ，加入边 $B_i \rightarrow D_i$ ，表示每个人都必须先出生后死亡。
2. 对每条“ P_i 死于 P_j 出生之前”的情报，加入边 $D_i \rightarrow B_j$ 。
3. 对每条“ P_i 与 P_j 的生命至少部分重叠”的情报，加入边 $B_i \rightarrow D_j$ 和 $B_j \rightarrow D_i$ 。这正对应于 P_i 在 P_j 死亡前出生，且 P_j 在 P_i 死亡前出生。

Algorithm 2

```

1: function CHECKCONSISTENCY( $n$ , type-1 facts, type-2 facts)
2:   Construct a directed graph  $H$  with vertices  $B_i, D_i$  for all  $1 \leq i \leq n$ 
3:   for each  $i = 1, 2, \dots, n$  do
4:     add edge  $B_i \rightarrow D_i$ 
5:   end
6:   for each type-1 fact  $(i, j)$  do
7:     add edge  $D_i \rightarrow B_j$ 
8:   end
9:   for each type-2 fact  $(i, j)$  do
10:    add edges  $B_i \rightarrow D_j$  and  $B_j \rightarrow D_i$ 
11:  end
12:   $L \leftarrow \text{TopologicalOrder}(H)$ 
13:  if  $L$  does not exist then
14:    return inconsistent
15:  end
16:  for each event  $x$  in  $L$  in order do
17:     $\tau(x) \leftarrow$  the position of  $x$  in  $L$ 
18:  end
19:  for each  $i = 1, 2, \dots, n$  do
20:    output  $(\tau(B_i), \tau(D_i))$ 
21:  end
22:  return the constructed dates
23: end

```

解:

若拓扑排序失败, 则 H 中存在有向环, 算法报告不一致; 若拓扑排序成功, 记 $\tau(x)$ 为事件节点 x 在拓扑序中的位置编号, 则对每个人 P_i 输出 $(\tau(B_i), \tau(D_i))$ 即得到一组候选出生和死亡日期。

下面证明算法正确。

先证必要性。若算法报告不一致, 则拓扑排序失败, 因此 H 中存在一个有向环 $x_1 \rightarrow x_2 \rightarrow \dots \rightarrow x_t \rightarrow x_1$ 。由边的语义可知, 沿环依次必须满足 $x_1 < x_2 < \dots < x_t < x_1$, 这与严格先后关系的反自反性矛盾。因此, 只要 H 中存在环, 就不存在任何一组出生和死亡日期能够同时满足全部情报。

再证充分性。设算法得到的拓扑序为 L , 并令 $\tau(x)$ 表示节点 x 在 L 中的位置。由拓扑序的定义, 对任意边 $(x, y) \in F$ 都有 $\tau(x) < \tau(y)$ 。于是:

1. 对每个 i , 由边 $B_i \rightarrow D_i$ 得 $\tau(B_i) < \tau(D_i)$, 故 P_i 一定先出生后死亡。
2. 对每条第一类情报, 由边 $D_i \rightarrow B_j$ 得 $\tau(D_i) < \tau(B_j)$, 故 P_i 的死亡时间早于 P_j 的出生时间。
3. 对每条第二类情报, 由边 $B_i \rightarrow D_j$ 和 $B_j \rightarrow D_i$ 得

$\tau(B_i) < \tau(D_j)$ 且 $\tau(B_j) < \tau(D_i)$ 。这正是两个生命区间 $[\tau(B_i), \tau(D_i)]$ 与 $[\tau(B_j), \tau(D_j)]$ 至少部分重叠的充要条件。

综上, 拓扑排序失败当且仅当原情报不一致; 若拓扑排序成功, 则按拓扑序中的位置编号赋值得到的日期一定满足全部约束, 因此算法正确。

最后分析时间复杂度。图中共有 $2n$ 个节点。若第一类情报有 k 条、第二类情报有 l 条, 则边数为 $n + k + 2l$ 。建图需要 $O(n + k + l)$ 时间, 拓扑排序需要 $O(|U| + |F|) = O(n + k + l)$ 时间, 最后输出所有人的出生和死亡日期需要 $O(n)$ 时间。因此算法总时间复杂度为 $O(n + k + l)$ 。